Applications of Recurrent Networks

Advanced Machine Learning

Sam Ogden

DRAFT

This slide deck is still under active development. Expect changes before it is finalized.

Learning objectives

- Explain how a language model writes text: start from `<bos>` and an empty state, feed each prediction back in, stop at `<eos>`
- Say why training feeds in the real previous word, but generating feeds in the model's own word
- Compare picking the most likely word with sampling
- Describe sequence to sequence: an encoder reads the input, and its final state starts the decoder
- Explain what the encoded state has to carry, and why one fixed-size state limits long inputs

Generating text

A language model, run in a loop

From predicting to writing

So far the model has always been handed **real text** and asked what comes next.

- At every step, the input was a real word from the data
- To write something new there is no real text, only a blank page
- What goes in at step 1? And at step 2, when there is no real first word?

The question:

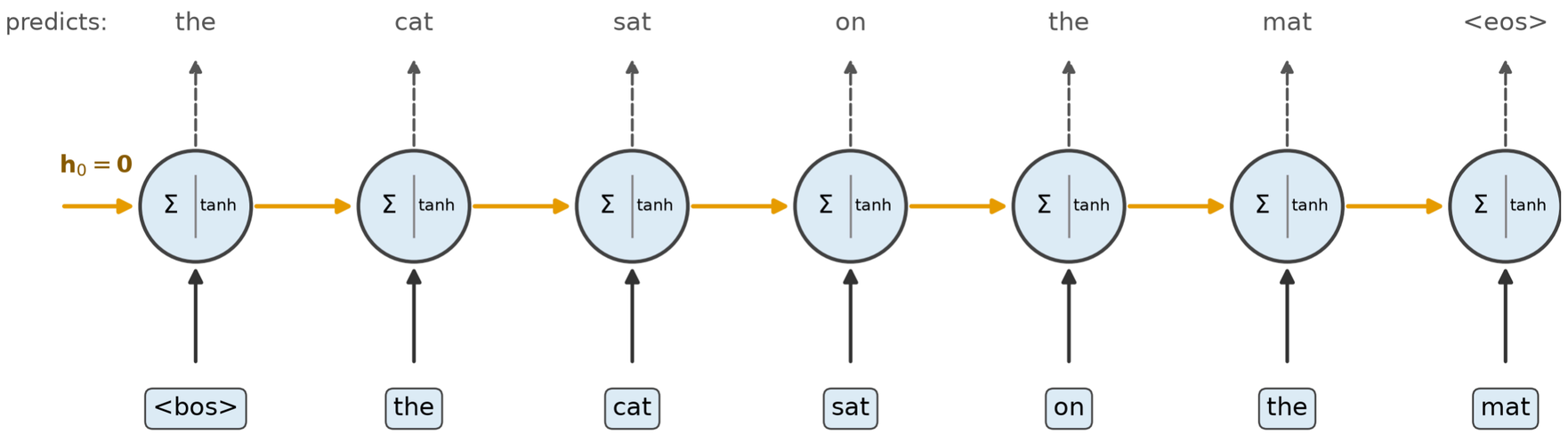
How do we get a whole sentence out of a model that only predicts one next word?

The Big Idea

Give the model only a start token, and feed each word it predicts back in as the next input, until it predicts an end token.

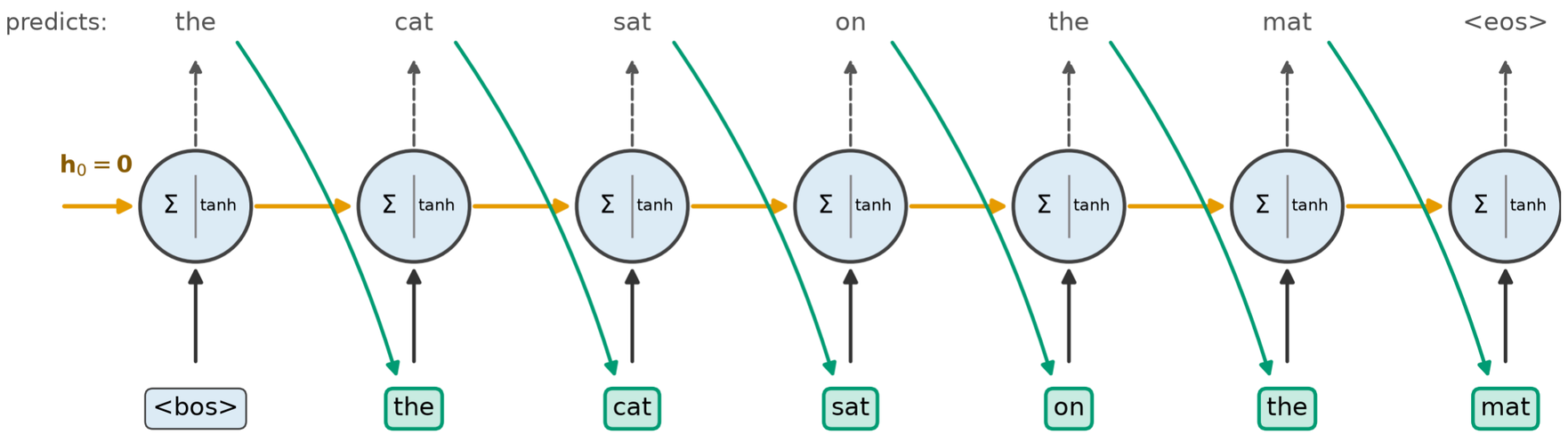
- The start and end tokens are the `<bos>` and `<eos>` markers from earlier
- The model is the one we already trained. Only where its inputs come from changes

Training: the real word goes in



- Blue: the input at every step is a **real** word from the training text, so we always know the right answer to compare with
- Every prediction is scored against the real next word. This is how we trained the language model

Generating: our own word goes in



- Two things start the loop: the token <bos> and an **empty state** (all zeros). Each prediction (green) is fed back in as the next input
- We stop when the model predicts <eos>. During training we feed the real word (called **teacher forcing**), but here there is no real text to feed

The loop with real numbers

Step	Input	Next-word probabilities	We pick
1	<bos>	the 1.00	**the**
2	the	cat 0.45, dog 0.26, horse 0.13	**cat**
3	cat	sat 0.48, ran 0.20, slept 0.17	**sat**
4	sat	on 1.00	**on**
5	on	the 1.00	**the**
6	the	mat 0.42, bed 0.22, floor 0.13	**mat**
7	mat	<eos> 1.00	**<eos>**

- A small GRU language model, trained on 600 made-up sentences. At each step we pick the **most likely** word, and we stop when it is sure it is done: <eos> at 1.00

The loop in PyTorch

```
@torch.no_grad()
def generate(model, max_len=20):
    token, state, words = BOS, None, []                          # start token, empty state
    for _ in range(max_len):
        states, state = model.rnn(model.embedding(token), state)  # one step of the RNN
        probs = torch.softmax(model.out(states[:, -1]), dim=-1)   # next-word probabilities
        token = probs.argmax(dim=-1, keepdim=True)                # pick a word (or sample one)
        if token.item() == EOS:
            break
        words.append(token.item())                                # the pick is also the next input
    return words
```

- `state=None` means all zeros, and the returned `state` is passed back in, so the memory carries over from step to step
- The only change from training: `token` is the model's own pick, not a word from the data

Picking the next word

At every step the model gives a probability for each possible next word. After “the”, it said: cat 0.45, dog 0.26, horse 0.13, and so on. Two ways to choose:

- **Most likely**: always take the highest, “cat”
- **Sample**: draw a word at random with those probabilities, so “cat” 45% of the time, “dog” 26% of the time, and so on
- The probabilities come from training: continuations that were common in the data get high probability, so sampling follows the **model**, not raw counts

Picking the next word: five runs

Five runs of the same trained model from `<bos>`. Every word in a run is picked the way the second column says.

Run	How we pick	Sentence written
1	Most likely	the cat sat on the mat
2	Sample	the horse sat on the garden
3	Sample	the cat sat on the mat
4	Sample	the cat slept on the mat
5	Sample	the dog ran on the mat

- Most likely gives the same sentence every run. Sampling gives variety, and some sentences are strange

Takeaway

A language model writes by predicting one word at a time and feeding each word back in.

Sequence to sequence

Starting the loop from what we read

A task with an input

Translate “the rabbit slept on the floor” into French.

- The generation loop starts from `<bos>` and an empty state, so on its own it writes **anything**
- Here the output has to depend on the input sentence
- The loop has two starting points, a token and a state. What can we change to tell it what to say?

The question:

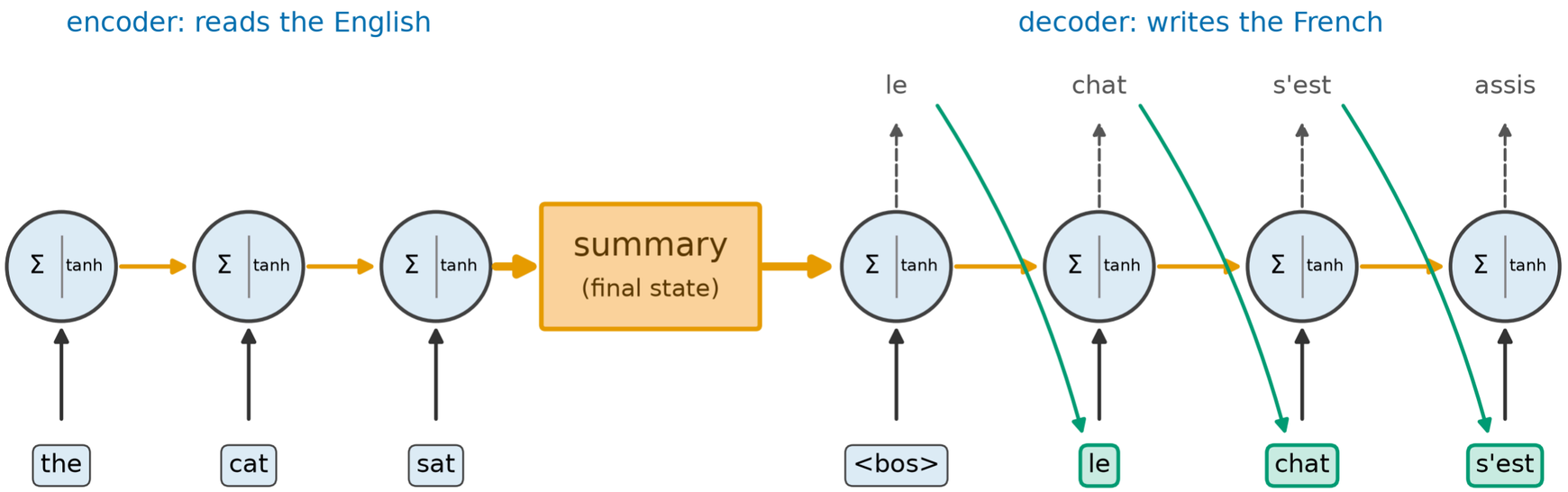
How do we make a generator write about one particular input?

The Big Idea

Read the input with one RNN, and start the generator from the final state of that RNN instead of from an empty state.

- That final state is a summary of the whole input, as in the sentiment classifier
- The generator is the language-model loop we just built. Only its starting state changes

Encoder and decoder



- The **encoder** reads the source and throws away its predictions. All that matters is its final state, the **summary** box
- The **decoder** is the generation loop. It starts from `<bos>` and that state, and feeds its own words back in

Same loop, different start

	Language model	Sequence to sequence
First input	<bos>	<bos>
Starting state	**All zeros (nothing read)**	**The encoder's summary of the input**
Each step	Predict the next word, feed it back in	Predict the next word, feed it back in
Stops when	It predicts <eos>	It predicts <eos>
Writes	Any sentence	A sentence about **this** input

- The state is the whole difference. It is the model's memory, and here it is **loaded** with the input before the first word is written

The start state steers the output

The decoder starts from	It writes
An empty state: 64 numbers, **all 0**	le chat a couru sur le jardin
64 numbers from the encoder, after reading “the rabbit slept on the floor”	le lapin a dormi sur le sol
64 numbers from the encoder, after reading “the pig jumped on the bridge”	le cochon a sauté sur le pont

- The same decoder every time. With an **empty state ($\mathbf{h_0 = 0}$)** nothing has been loaded, so it writes a typical French sentence: it is a language model of French
- Start it from the encoder’s state and it writes the sentence that state describes. So that one vector has to carry the meaning of the input

A small translation model we trained

English (unseen)	Model's French	Right?
the pig jumped on the bridge	le cochon a sauté sur le pont	yes
the rabbit slept on the floor	le lapin a dormi sur le sol	yes
the dog played on the garden	le chien a joué sur le toit	no, it is jardin

- **We trained this model ourselves**, a GRU encoder and decoder, on 140 English-French sentence pairs from a tiny made-up language
- On 40 new combinations of words it knew but never saw together, **88%** were written exactly right (80% to 95% over five runs)
- The errors are usually the place word, the part the state carries least reliably

Sequence to sequence in PyTorch

```
 1 class Seq2Seq(nn.Module):
 2     def __init__(self, src_vocab, tgt_vocab, dim, hidden):
 3         super().__init__()
 4         self.src_embedding = nn.Embedding(src_vocab, dim)
 5         self.encoder = nn.GRU(dim, hidden, batch_first=True)
 6         self.tgt_embedding = nn.Embedding(tgt_vocab, dim)
 7         self.decoder = nn.GRU(dim, hidden, batch_first=True)
 8         self.out = nn.Linear(hidden, tgt_vocab)
 9
10     def forward(self, src, tgt_in):
11         _, state = self.encoder(self.src_embedding(src))              # read the input
12         states, _ = self.decoder(self.tgt_embedding(tgt_in), state)   # start from its state
13         return self.out(states)
```

- The second thing an `nn.GRU` returns is its final state. The second thing it takes is the state to start from
- To generate, run the same loop as before, but start `state` from the encoder's final state instead of `None`

Training a sequence-to-sequence model

```
logits = model(src, tgt_in)                    # encoder, then decoder, in one forward pass
loss = F.cross_entropy(logits.flatten(0, 1), tgt_out.flatten())
loss.backward()                                # one loss, gradients reach both halves
optimizer.step()
```

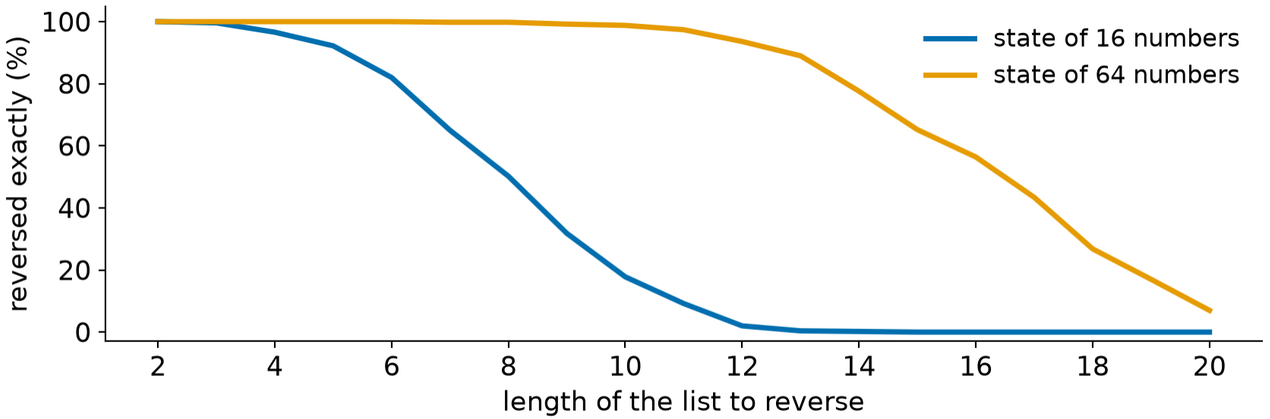
- The two halves are trained **together**, as one network, on pairs: an English sentence and its French translation
- The decoder is fed the real previous French word (teacher forcing) and scored on the next one
- The gradient flows back through the starting state into the encoder, so the encoder learns what the summary must contain
- Trained separately, the encoder would have no target of its own. Its “right answer” is whatever the decoder finds useful

Anything can prime the state

Input	Encoder	Decoder writes	Trained on pairs of
English sentence	RNN	French sentence	English and French
A long article	RNN	A short summary	Articles and summaries
A question	RNN	An answer	Questions and answers
An image	CNN	A caption	Images and captions

- The decoder can write about anything we can **load into its starting state**, such as a CNN's feature vector
- Each row is a **separate model**: encoder and decoder trained together on that task's pairs

One state has to carry the whole input



- A small test: read a list of digits, then write it backwards. The encoder's final state is the **only** thing the decoder gets
- Longer lists fail, and a bigger state lasts longer but also fails. Everything about the input has to fit in one fixed-size vector

Takeaway

A sequence-to-sequence model is a language model whose starting state is a summary of the input.

Summary

Recap: using a recurrent network

Use	Input	Starting state	What we use
Classify a sequence	The whole text	Zeros	The last state
Language model (training)	The real text	Zeros	A prediction at every step
Generate text	`<bos>`, then its own words	Zeros	Each prediction, fed back in
Sequence to sequence	`<bos>`, then its own words	**The encoder's summary**	Each prediction, fed back in

Recap: key ideas

- To **generate**, start from <bos> and an empty state, feed each predicted word back in, and stop at <eos>. Training feeds the real word instead (teacher forcing)
- Picking the most likely word gives the same sentence every time. Sampling gives variety
- **Sequence to sequence**: an encoder reads the input, and its final state starts the decoder. The state is what steers the output
- Anything that can be turned into a vector can start a decoder: a sentence, a document, an image
- One fixed-size state has to hold the whole input, and longer inputs break it

Where this goes next

- Attention: let the decoder look back at **every** step of the encoder, not only its final state
- That idea, with the recurrence removed, is the transformer
- Not covered here: beam search, bidirectional and stacked RNNs, and how translation is scored